

Software Design and Coding Standards; C and Python

H. Bowman

October 3, 2026

Contents

1	Introduction	5
2	Scope	5
3	Terminology	5
4	General Design Principles	5
5	Requirements and Rule Priority	5
I	C Programming Standard	5
5.1	General Principles	6
5.2	C11 Language Subset	6
5.2.1	Permitted Language	6
5.2.2	Prohibited or Restricted Features	6
5.3	Formatting	7
5.3.1	Indentation	7
5.3.2	Alignment	7
5.3.3	Braces	7
5.3.4	Whitespace	7
5.3.5	Line Length	8
5.3.6	Automated Formatting	8
5.4	Naming	8
5.4.1	General Naming Requirements	8
5.4.2	Functions	9
5.4.3	Variables	9
5.4.4	Types	9
5.4.5	Constants and Macros	9
5.4.6	Private Names	9

5.5	Variables	10
5.5.1	General Variable Requirements	10
5.6	Module Boundaries	10
5.6.1	Module Structure	10
5.6.2	Header Independence	10
5.6.3	Include Ownership	10
5.6.4	Circular Dependencies	10
5.7	Visibility	11
5.7.1	Public Declarations	11
5.7.2	Private Declarations	11
5.7.3	Global Variables	11
5.7.4	Opaque Types	11
5.8	Type Safety	12
5.8.1	Fixed-Width Integer Types	12
5.8.2	Boolean Values	12
5.8.3	Enumerations	12
5.8.4	Explicit Conversions	12
5.8.5	Const Correctness	13
5.8.6	Additional Rules	13
5.9	Initialization	13
5.9.1	General Requirement	13
5.9.2	Default Initialization	13
5.9.3	Structures	13
5.9.4	Arrays	14
5.9.5	Pointers	14
5.9.6	Static Storage	14
5.9.7	Initialization Functions	14
5.9.8	Partial Initialization	14
5.10	Control Flow	14
5.10.1	Control Flow	14
5.10.2	Loops	15
5.11	Functions	15
5.11.1	Function Requirements	15
5.11.2	Parameter Validation	15
5.11.3	Return Values	15
5.12	Memory Safety	16
5.12.1	General Requirement	16

5.12.2	Allocation Policy	16
5.12.3	Allocation Arithmetic	17
5.12.4	Ownership	17
5.13	Arrays and Strings	17
5.13.1	Array Bounds	17
5.13.2	String Handling	17
5.13.3	Binary Data	17
5.14	Pointers	18
5.15	Integer Safety	18
5.16	Error Handling	18
5.17	Preprocessor	18
5.18	External Libraries	18
5.18.1	Library Policy	18
5.18.2	Approval Requirements	18
5.18.3	Wrapper Requirement	18
5.19	Concurrency	19
5.19.1	Applicability	19
5.19.2	Shared State	19
5.19.3	Ownership	19
5.19.4	Mutexes	19
5.19.5	Lock Ordering	19
5.19.6	Atomics	20
5.19.7	Interrupts and Signal Handlers	20
5.19.8	Thread Cancellation	20
5.19.9	Testing	20
5.20	Documentation	20
5.20.1	Documentation Requirements	20
5.20.2	Public Interfaces	20
5.20.3	Module Documentation	21
5.20.4	Comments	21
5.20.5	TODO Comments	22
5.20.6	Generated Documentation	22
5.21	Tooling	22
5.21.1	Compiler	22
5.21.2	Analysis	22
5.21.3	Required Build Checks	23
5.22	Exceptions	23

5.22.1 Purpose	23
5.22.2 When an Exception Is Required	23
5.22.3 Exception Record	23
5.22.4 Warning Suppression	24
5.22.5 Review	24
II Python Programming Standard	24
5.23 Python Formatting	24
5.23.1 Formatting Rules	24
5.24 Python Naming	25
5.25 Python Imports	25
5.26 Type Hints	25
5.26.1 Type Hints	25
5.27 Functions and Classes	25
5.28 Error Handling	25
5.29 Testing	25
5.30 Python Review Checklist	25
III Tooling and Verification	25
IV Development Process	25
A C Review Checklist	25
B Python Review Checklist	26
C Release Checklist	26

- 1 Introduction**
- 2 Scope**
- 3 Terminology**
- 4 General Design Principles**
- 5 Requirements and Rule Priority**

Part I

C Programming Standard

This project uses a restricted subset of ISO C11 intended to improve memory safety, type safety, portability, analyzability, and maintainability.

The C11 standard is organized into the following areas:

- general principles;
- language subset;
- formatting;
- naming;
- Variables;
- module boundaries;
- public and private visibility;
- type safety;
- initialization;
- control flow;
- functions;
- memory safety;
- arrays and strings;
- pointers;
- integer safety;
- error handling;
- preprocessor usage;
- external libraries;
- concurrency;
- documentation and verification.

5.1 General Principles

Clarity First: Code should be as readable as prose, spoken aloud.

Simplicity is Beauty: Short, understandable code reduces bugs; remove dead code and refactor duplicates into functions.

Consistent Style: Align with existing project style; use format-conversion tools if needed.

5.2 C11 Language Subset

5.2.1 Permitted Language

Source code shall conform to ISO C11. Compiler extensions shall not be used unless they are isolated behind a documented platform abstraction.

Rules prohibit unrestricted control flow and recursion. Instead it requires statically bounded loops, restricted runtime allocation, requires return-value checks, limit the preprocessor use, restrict the use of pointers, and require warning-free compilation and static analysis.

5.2.2 Prohibited or Restricted Features

The following features require documented justification:

- unions;
- bit-fields;
- flexible array members;
- volatile objects;
- atomics;
- function pointers;
- implementation-defined behavior;
- compiler extensions;
- direct hardware access;
- dynamic memory allocation.

The following features are prohibited:

- `goto`;
- `setjmp`;
- `longjmp`;
- direct recursion;
- indirect recursion;
- variable-length arrays;
- implicit function declarations;
- implicit integer-to-pointer conversions;

- implicit pointer-to-integer conversions;
- function-like macros where an inline function is practical;
- compiler-specific types in portable modules;
- unbounded loops in deterministic modules;
- unrestricted global mutable state.

5.3 Formatting

5.3.1 Indentation

Tabs shall be used for indentation. Spaces shall not be used to represent indentation.

One tab shall represent one logical indentation level. The display width of a tab shall be four spaces in the project editor configuration.

5.3.2 Alignment

Spaces may be used for alignment within a line, but alignment shall not create a requirement for additional whitespace on later lines.

5.3.3 Braces

Braces shall always be used for compound statements, including statements containing only one instruction.

Opening braces shall appear on the same line as the associated statement.

```
if (condition) {
    process_item();
} else {
    handle_error();
}
```

5.3.4 Whitespace

There shall be spaces:

- around binary operators;
- after commas;
- between a type and a variable name where appropriate.

After every keyword (if, while, etc) of the C language, a space must be added. e.g.

```
if (i == 1)
while (i == 0)
```

keywords include:

- if
- switch

- case
- for
- do
- while

but not:

- sizeof
- typeof
- alignof
- attribute

There shall be no trailing whitespace.

5.3.5 Line Length

Lines should not exceed 100 characters. A longer line is permitted only when breaking it would reduce readability.

5.3.6 Automated Formatting

All C files shall be processed using the project-approved formatter configuration. Manual formatting preferences shall not override the project formatter.

Package indent under Debian and Ubuntu Linux distributions is recommended for formatting C code. On windows it can be found here: <https://gnuwin32.sourceforge.net/packages/indent.htm>

5.4 Naming

5.4.1 General Naming Requirements

Names shall describe the purpose, behavior, or ownership of the object they identify. Abbreviations shall be avoided unless they are widely understood within the project.

Names shall:

- use lowercase **snake_case** for functions and variables;
- use uppercase **UPPER_SNAKE_CASE** for macros;
- use descriptive names for types;
- distinguish counts, sizes, indexes, capacities, and offsets;
- avoid names that differ only by capitalization;
- avoid names that shadow variables in an enclosing scope;
- avoid single-letter names except for short loop indexes;
- avoid misleading names.

5.4.2 Functions

Functions shall use verb-based names that describe their operation.

```
buffer_init();
buffer_write();
buffer_clear();
packet_is_valid();
```

Boolean-returning functions should use names such as:

```
is_valid();
has_error();
can_write();
```

5.4.3 Variables

Variables shall describe their content and, where useful, their unit.

```
size_t packet_length;
uint32_t timeout_ms;
uint8_t retry_count;
bool connection_is_open;
```

Names such as `data`, `value`, `temp`, and `result` should be avoided unless their meaning is clear from the local context.

5.4.4 Types

Project-defined types shall use a consistent project convention.

```
typedef struct buffer buffer_t;
typedef enum device_state device_state_t;
```

Structure tags and typedef names shall not unnecessarily duplicate one another.

5.4.5 Constants and Macros

Constants should preferably be represented using typed constants, enumerations, or `static const` objects instead of macros.

```
static const size_t max_packet_length = 1024U;
```

Macros shall use uppercase names with a project-specific prefix.

```
#define PROJECT_MAX_PACKET_LENGTH 1024U
```

5.4.6 Private Names

Private file-scope functions and variables shall use `static`. A project prefix may be used for private names when required to avoid collisions with external symbols.

5.5 Variables

5.5.1 General Variable Requirements

Group local variables together by type.

Do not declare variables after first executable statement.

Each variable has a single purpose; avoid globals.

Prevent name clashes, respect byte order in communication structs, and never use uninitialized variables.

5.6 Module Boundaries

5.6.1 Module Structure

Each module shall consist of:

- one public header file, when an external interface is required;
- one source file;
- private declarations contained in the source file;
- unit tests for externally visible behavior.

A module shall have one primary responsibility.

5.6.2 Header Independence

Every public header shall compile independently when included into an otherwise empty translation unit.

```
#include "buffer.h"

int main(void)
{
    return 0;
}
```

5.6.3 Include Ownership

A source file shall include the header that declares its public interface before other project headers.

```
#include "buffer.h"

#include <stddef.h>
#include <stdint.h>

#include "memory.h"
```

5.6.4 Circular Dependencies

Circular dependencies between modules are prohibited.

Modules shall depend on abstractions or narrow interfaces rather than including unrelated internal headers.

5.7 Visibility

5.7.1 Public Declarations

A public declaration is any declaration exposed through a module's interface header.

Public declarations shall be limited to:

- functions intended for use by other modules;
- opaque object types;
- public enumerations;
- public constants;
- public error codes;
- public configuration structures when required by the interface.

5.7.2 Private Declarations

Private functions, variables, constants, and types shall be defined in the corresponding source file.

Private file-scope declarations shall use the `static` specifier.

```
static uint32_t calculate_checksum(const uint8_t *data,
                                   size_t length);

static uint32_t checksum_seed = 0U;
```

5.7.3 Global Variables

Global mutable variables are prohibited.

File-scope variables shall be private unless there is a documented architectural reason for making them public.

Public variables shall not be used for ordinary module communication. Functions shall be used instead.

5.7.4 Opaque Types

Public headers should expose opaque structures rather than structure definitions.

```
/* buffer.h */

#ifndef PROJECT_BUFFER_H
#define PROJECT_BUFFER_H

#include <stddef.h>
#include <stdint.h>

typedef struct buffer buffer_t;

buffer_t *buffer_create(uint8_t *storage, size_t capacity);
int buffer_write(buffer_t *buffer,
                 const uint8_t *data,
                 size_t length);
size_t buffer_size(const buffer_t *buffer);

#endif
```

The structure definition remains private:

```
/* buffer.c */

struct buffer {
    uint8_t *storage;
    size_t capacity;
    size_t length;
};
```

5.8 Type Safety

5.8.1 Fixed-Width Integer Types

Fixed-width integer types shall be used when the width or signedness is important.

```
#include <stdint.h>

uint32_t packet_length;
int32_t temperature;
uint8_t status;
```

The types `int8_t`, `uint8_t`, and similar types shall be used instead of assuming the size of `char`, `short`, or `int`.

5.8.2 Boolean Values

Boolean values shall use `bool` from `<stdbool.h>`.

```
#include <stdbool.h>

bool packet_is_valid;
```

Integer values shall not be used as booleans unless the conversion is explicit and documented.

5.8.3 Enumerations

Enumerations shall be used for finite sets of named states.

```
typedef enum {
    MODE_IDLE = 0,
    MODE_ACTIVE,
    MODE_ERROR
} mode_t;
```

5.8.4 Explicit Conversions

Conversions that may change signedness, width, precision, or interpretation shall be explicit.

```
uint32_t value = 0U;
uint16_t reduced_value = (uint16_t)value;
```

The conversion shall be safe. A cast shall not be used merely to silence a compiler warning.

5.8.5 Const Correctness

The `const` qualifier shall be used whenever an object is not modified through a pointer.

```
void process_packet(const uint8_t *data, size_t length);
```

5.8.6 Additional Rules

- Do not compare signed and unsigned values without an explicit conversion or documented reason.
- Do not use plain `char` for binary data.
- Do not use `float` or `double` for exact quantities.
- Do not rely on the size of `int`.
- Do not use casts to convert unrelated pointer types.
- Do not use type-punning through incompatible pointer types.
- Do not use an enumeration outside its defined semantic domain.

5.9 Initialization

5.9.1 General Requirement

Every object shall be initialized before it is read.

Initialization shall occur as close as practical to the declaration.

```
uint32_t retry_count = 0U;  
bool packet_is_valid = false;
```

Uninitialized local variables are prohibited.

5.9.2 Default Initialization

Objects shall be initialized to a valid and documented default state.

```
struct device device = {  
    .state = DEVICE_STATE_OFF,  
    .timeout_ms = 1000U,  
    .error_code = ERROR_NONE  
};
```

Designated initializers shall be used for structures when they improve clarity or protect against changes in member ordering.

5.9.3 Structures

Structure members shall be initialized explicitly when their zero value is not the required default.

The following form is preferred for complete initialization:

```
struct configuration configuration = {  
    .enabled = true,  
    .retry_limit = 3U,  
    .timeout_ms = 500U  
};
```

5.9.4 Arrays

Arrays shall be initialized explicitly when partial initialization could hide an error.

```
uint8_t header[HEADER_LENGTH] = {0};
```

5.9.5 Pointers

Pointers shall be initialized to a valid object or to `NULL`.

```
uint8_t *buffer = NULL;
```

A pointer shall not be dereferenced unless its validity has been established.

5.9.6 Static Storage

Static and file-scope objects shall have an explicit initializer when the initial state is important to the module contract.

5.9.7 Initialization Functions

Initialization functions shall:

- be called before any other operation on the object;
- be safe to call only according to their documented contract;
- establish all required invariants;
- report failure explicitly;
- leave the object in a documented state when initialization fails.

```
int device_init(device_t *device);
```

5.9.8 Partial Initialization

An object shall not be exposed to other modules until initialization is complete.

If initialization requires multiple steps, cleanup shall be defined for each partially initialized state.

5.10 Control Flow

5.10.1 Control Flow

The following are prohibited:

- `goto`;
- `setjmp`;
- `longjmp`;
- recursion;
- deeply nested conditionals;
- hidden control flow in macros.

5.10.2 Loops

Loops shall have a clear termination condition.

Loops in deterministic or safety-critical code shall have a statically reviewable upper bound.

```
for (size_t index = 0U; index < MAX_ITEMS; ++index) {  
    process_item(items[index]);  
}
```

A loop controlled by external input shall still have a maximum iteration limit.

5.11 Functions

5.11.1 Function Requirements

Functions shall:

- have one primary responsibility;
- have a clearly defined contract;
- validate input parameters;
- return an explicit status for recoverable failure;
- avoid hidden side effects;
- avoid modifying caller data unless documented;
- remain short enough for complete review.
- Limit nesting depth to four levels.

Functions should normally have no more than 10 local variables. If more are required, the function should be refactored into smaller functions.

Every function that may be accessed from outside its module **MUST** include a function prototype (declaration).

Function declarations and definitions that accept no parameters **MUST** explicitly use (void), never empty parentheses ().

Function implementations **MUST** place the return type and any storage-class or qualifier keywords on a separate line from the function name.

A function **SHOULD** have at most 2 return points: one early return at the top of the function for parameter/argument validation, and one return at the end of the function. Avoid return statements in the middle of the function body; use a return-value variable to accumulate the result instead.

5.11.2 Parameter Validation

Public functions shall validate all pointer parameters, lengths, ranges, and state assumptions that are not guaranteed by the type system.

5.11.3 Return Values

The caller shall check every non-void return value.

If a return value is intentionally ignored, the reason shall be explicit.

```
(void)log_message("startup complete");
```

5.12 Memory Safety

5.12.1 General Requirement

Code shall not produce:

- out-of-bounds accesses;
- use-after-free;
- double-free;
- invalid frees;
- null-pointer dereferences;
- uninitialized reads;
- memory leaks;
- access through a dangling pointer;
- writes through a read-only object;
- integer-overflow-derived allocation sizes.

For maximum safety general rules include:

- no heap allocation in core logic;
- no heap allocation unless needed;
- caller-provided buffers where possible;
- explicit buffer lengths;
- no hidden ownership transfer;
- no ownership implied only by naming;
- cleanup paths tested.

5.12.2 Allocation Policy

Static allocation and caller-provided storage are preferred.

Dynamic allocation shall be avoided in ordinary modules and is prohibited after initialization in deterministic or safety-critical modules.

Every allocation shall have:

- one owner;
- one documented release path;
- a checked failure path;
- a defined maximum size;
- a test for boundary conditions.

5.12.3 Allocation Arithmetic

Allocation-size arithmetic shall be checked before allocation.

```
if (count > SIZE_MAX / sizeof(*items)) {  
    return ERROR_SIZE_OVERFLOW;  
}  
  
items = malloc(count * sizeof(*items));  
if (items == NULL) {  
    return ERROR_OUT_OF_MEMORY;  
}
```

5.12.4 Ownership

Functions shall document whether pointer parameters are:

- borrowed and not retained;
- owned by the caller;
- transferred to the callee;
- returned with ownership transferred to the caller.

5.13 Arrays and Strings

5.13.1 Array Bounds

Every array access shall be demonstrably within bounds.

A pointer to an array shall be accompanied by its length unless the length is guaranteed by the interface contract.

```
int process_bytes(const uint8_t *data, size_t length);
```

5.13.2 String Handling

The following functions are prohibited:

- `gets`;
- `strcpy`;
- `strcat`;
- `sprintf`;
- `scanf` without a field-width limit.

String operations shall specify destination capacity and shall preserve null termination.

5.13.3 Binary Data

Binary buffers shall use `uint8_t` and an explicit length. They shall not be treated as null-terminated strings.

5.14 Pointers

5.15 Integer Safety

5.16 Error Handling

5.17 Preprocessor

5.18 External Libraries

5.18.1 Library Policy

The project shall depend only on:

- the ISO C11 standard library;
- operating-system interfaces required by the target;
- project-owned libraries;
- third-party libraries approved in the dependency register.

A third-party library shall not be added solely for convenience when the required functionality can be implemented safely using C11 or existing project code.

5.18.2 Approval Requirements

Each external library shall have:

- a documented purpose;
- a version pinned in the build system;
- a license record;
- a maintenance status;
- a security review;
- a documented public API surface;
- a removal or replacement plan where practical.

5.18.3 Wrapper Requirement

External libraries shall be accessed through a project-owned wrapper module.

Application modules shall not include third-party headers directly unless the dependency is explicitly approved as part of the public architecture.

```
/* project_json.h */

#ifndef PROJECT_JSON_H
#define PROJECT_JSON_H

int project_json_parse(const char *text);

#endif
```

5.19 Concurrency

5.19.1 Applicability

Concurrency includes operating-system threads, interrupt handlers, callbacks, signal handlers, asynchronous tasks, and hardware event handlers.

Concurrency shall not be introduced unless it is required by the system design.

5.19.2 Shared State

Shared mutable state shall be minimized.

Every shared object shall have a documented synchronization strategy.

The following are prohibited:

- undocumented shared mutable state;
- data races;
- unsynchronized access to shared objects;
- relying on timing for correctness;
- using `volatile` as a replacement for synchronization.

5.19.3 Ownership

Each shared object shall have one of the following documented policies:

- protected by a mutex;
- accessed only by one thread;
- accessed atomically;
- immutable after initialization;
- exchanged through a documented message-passing interface.

5.19.4 Mutexes

Mutex acquisition and release shall be visibly paired.

```
mutex_lock(&state_mutex);  
  
update_shared_state();  
  
mutex_unlock(&state_mutex);
```

A mutex shall not be held while calling an external function unless the locking contract explicitly permits it.

5.19.5 Lock Ordering

When multiple locks are required, the project shall define a global lock ordering.

Locks shall always be acquired in that order to prevent deadlock.

5.19.6 Atomics

C11 atomic objects shall be used only when their memory-ordering behavior is understood and documented.

The default sequentially consistent ordering shall be preferred unless a different ordering is justified.

5.19.7 Interrupts and Signal Handlers

Signal handlers and interrupt handlers shall perform minimal work.

They shall not call functions that are not safe in their execution context.

Communication with the main application shall use an approved mechanism, such as an atomic flag, queue, or event object.

5.19.8 Thread Cancellation

Thread cancellation shall not be used unless cleanup behavior is fully defined.

Resources shall not be leaked when a thread exits or is cancelled.

5.19.9 Testing

Concurrent code shall be tested under:

- repeated execution;
- increased scheduling variability;
- thread and lock analysis;
- race detection where supported;
- sanitizer-enabled builds where supported.

5.20 Documentation

5.20.1 Documentation Requirements

Documentation shall explain intent, contracts, assumptions, ownership, and limitations. It shall not merely repeat the implementation.

Documentation shall be updated whenever behavior or interfaces change.

5.20.2 Public Interfaces

Every public function shall document:

- its purpose;
- each parameter;
- the return value;
- possible error conditions;

- ownership and lifetime rules;
- thread-safety requirements;
- whether pointers may be NULL.

```
/**
 * @brief Writes bytes to a buffer.
 *
 * @param buffer Buffer that receives the data.
 * @param data Input data. The memory is borrowed and is not retained.
 * @param length Number of bytes to write.
 *
 * @return 0 on success.
 * @return -1 if an argument is invalid.
 * @return -2 if insufficient capacity is available.
 *
 * @threadsafe No. The caller shall provide synchronization.
 */
int buffer_write(buffer_t *buffer,
                 const uint8_t *data,
                 size_t length);
```

5.20.3 Module Documentation

Each module shall document:

- its responsibility;
- its public interface;
- its dependencies;
- its ownership model;
- its initialization requirements;
- its thread-safety model;
- its error-reporting strategy.

5.20.4 Comments

Comments shall explain why the code exists, not simply describe what the code visibly does.

Poor comment:

```
// Increment index
index++;
```

Useful comment:

```
// The hardware requires a delay after reset before the status register becomes valid.
delay_ms(HARDWARE_RESET_DELAY_MS);
```

For multi-line comments use space+asterisk for every line

5.20.5 TODO Comments

TODO comments shall include an issue identifier or owner.

```
// TODO(PROJ-142): Replace polling with event notification.
```

Unowned TODO comments shall not be committed to production code.

5.20.6 Generated Documentation

Public interfaces should use a consistent documentation format, such as Doxygen. Generated documentation shall be treated as a build artifact and shall not replace source-level documentation.

5.21 Tooling

5.21.1 Compiler

The compiler shall be configured for C11:

```
-std=c11
```

The following warning categories should be enabled for GCC or Clang:

```
-Wall  
-Wextra  
-Wpedantic  
-Wconversion  
-Wsign-conversion  
-Wshadow  
-Wcast-qual  
-Wcast-align  
-Wstrict-prototypes  
-Wmissing-prototypes  
-Wformat=2  
-Wundef  
-Werror
```

The exact warning set may vary by compiler, but all warnings shall be resolved or formally justified.

5.21.2 Analysis

The project shall use:

- compiler warnings;
- a static analyzer;
- AddressSanitizer during testing;
- UndefinedBehaviorSanitizer during testing;
- a formatter;
- unit tests;
- boundary and negative tests.

5.21.3 Required Build Checks

A merge shall be rejected if:

- formatting fails;
- compilation produces warnings;
- static analysis produces unreviewed findings;
- sanitizers detect an error;
- unit tests fail;
- coverage or safety requirements are not met.

5.22 Exceptions

5.22.1 Purpose

Exceptions provide a controlled method for deviating from this standard. They shall not be used to avoid normal engineering review.

5.22.2 When an Exception Is Required

An exception is required when code:

- uses a prohibited language feature;
- introduces an unapproved external dependency;
- exposes a normally private symbol;
- uses dynamic allocation in a restricted module;
- suppresses a compiler or static-analysis warning;
- violates a mandatory formatting or naming rule;
- requires implementation-defined or compiler-specific behavior;
- cannot satisfy a memory, type, or concurrency rule.

5.22.3 Exception Record

Each exception shall document:

- the rule identifier;
- the affected file, module, or function;
- the reason for the deviation;
- the risks introduced;
- the mitigation or compensating control;
- the approving reviewer;

- the date of approval;
- the planned review or expiration date.

Field	Value
Rule identifier	C-MEM-004
Affected module	<code>network_buffer.c</code>
Deviation	Dynamic allocation is used after initialization.
Reason	Incoming packet size is not known during startup.
Mitigation	Maximum allocation is bounded and failure is handled.
Approver	Technical lead
Review date	2026-10-03
Status	Approved

5.22.4 Warning Suppression

Warning suppression shall be localized to the smallest possible scope.

A suppression shall include a comment explaining:

- which warning is suppressed;
- why it is safe;
- why the code cannot be rewritten to avoid it.

Suppressing warnings globally is prohibited.

5.22.5 Review

Exceptions shall be reviewed when the affected module changes and at each major release.

Expired or no-longer-needed exceptions shall be removed.

Part II

Python Programming Standard

This part defines the coding and design requirements for Python source code.

5.23 Python Formatting

5.23.1 Formatting Rules

Python code shall follow PEP 8 and the project formatter configuration.

- Four spaces shall be used for indentation.
- Tabs shall not be used.
- Lines should not exceed 88 characters.

- Black shall be used for automatic formatting.
- Ruff shall be used for linting.

```
def calculate_checksum(data: bytes) -> int:
    """Return the checksum for a byte sequence."""
    return sum(data) % 256
```

5.24 Python Naming

5.25 Python Imports

5.26 Type Hints

5.26.1 Type Hints

Public functions and methods shall include type hints for parameters and return values.

```
def load_configuration(path: str) -> dict[str, str]:
    """Load configuration values from a file."""
    ...
```

Static type checking should be performed using `mypy` or the project-approved equivalent.

5.27 Functions and Classes

5.28 Error Handling

5.29 Testing

5.30 Python Review Checklist

Part III

Tooling and Verification

Part IV

Development Process

A C Review Checklist

- ☐ C-REV-001 All compiler warnings are resolved.
- ☐ C-REV-002 All return values are handled.
- ☐ C-REV-003 Pointer ownership is documented.
- ☐ C-REV-004 Memory allocation failures are handled.
- ☐ C-REV-005 Loop bounds are reviewable.

- ☐ C-REV-006 Static-analysis warnings are resolved.
- ☐ C-REV-007 Unit tests cover boundary conditions.
- ☐ C-REV-008 The code follows the project formatter.

B Python Review Checklist

- ☐ PY-REV-001 The code is formatted with Black.
- ☐ PY-REV-002 Ruff reports no unresolved violations.
- ☐ PY-REV-003 Public functions have type hints.
- ☐ PY-REV-004 Public interfaces have docstrings.
- ☐ PY-REV-005 Exceptions are handled explicitly.
- ☐ PY-REV-006 External resources use context managers.
- ☐ PY-REV-007 Tests cover invalid and boundary inputs.

C Release Checklist

References

- [1] Liangxu Linux. Essential c coding standards for clear, safe, and maintainable code, January 2025.
- [2] MaJerle. C code style, 2018.